Chapter 2: Quantum Algorithmic Foundations

Content:

1. Two examples: integer factorizations and computing GCDs
2. Measuring computational cost
3. Classical computation on quantum computers

1. Interger factorization

Problem:

> Input: integer $n \geq 2$ Output: The prime factorization of N

See fundamental theorem of arithmetic ex : $12 = 2^3.3$

GCD: Greatest Common Divisor

> Input: Two positives integers N and M Output: The greatest common divisor of N and M.

Abstract view of computaiton:

$$input \rightarrow computation \rightarrow output$$

2. Measuring computational Cost

Inputs are booleans string
The computation could be modeled in a varierty of ways, including : (but not limited) these:

- turing machine
- boolean circuits
- quantum circuits
- python programs

Lenght of the binary encoding of N:

$$lg(N) = \begin{cases} 1 & \text{if } N = 0 \\ 1 + \lfloor log_2(N) \rfloor & \text{if } N \geq 1 \end{cases} \tag{1}$$

Several objects can be encoded as binary strings. In general, the length of the input is the length of the input binary string.

Elementary operations: for a circuit based on computational models, it is typical to view gates as elementary operations.

A standard single qubit unitary gates:

- $X, Y, Z, H, S, S^\dagger, T, T^\dagger$
- Controlled-Not Gates
- Single qubit standard basis measurements

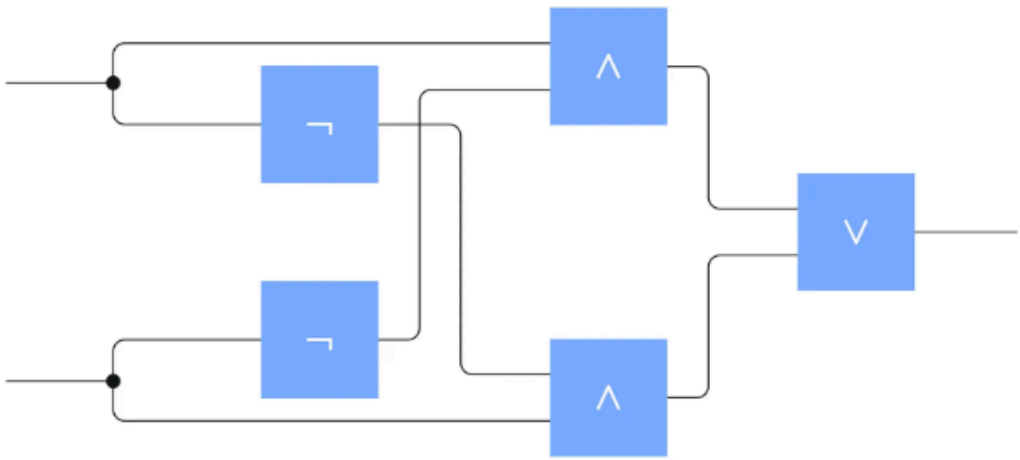
These unitary operations are universal; all other operations can be approximated with the above gates.

A standard Boolean gate set:

- AND
- OR
- NOT
- $FANOUT$

$FANOUT$ gates are not always considered gates, but here it is interesting to consider them as such.

The size of a Boolean circuit is the total number of gates it includes. We use size(C) to refer to the size of circuit C.



Here, the circuit size is 7, because there are 5 logic gates and 2 $FANOUT$s, giving a total of 7 gates.

The size of a circuit corresponds to the sequential running time. We will calculate the computational cost as follows in this lesson.

Circuit depth:

The depth of a circuit is the maximum number of gates encountered on any path from the input to the output. In the above case, the depth of the circuit is 4 (1 $FANOUT$ and 3 gates). The circuit depth corresponds to the parallel running time.

Cost of a function with an input length:

Each circuit has a fixed size, so we need a family $C_1, C_2, \ldots$ of circuits to describe the algorithm, typically one circuit for each input length.

<u>Example</u>:

A classic algorithm for integer factorization can be described by a family of Boolean circuits, where C_n factors n-bit numbers.

The cost of such an algorithm is described by a function:

$$t(n) = \text{size}(C_n)$$

Asymptotic notation:

Big O notation:

For two functions $g(n)$ and $h(n)$, we note: $g(n) = O(h(n))$ if there exists a positive real integer $c > 0$ and a positive integer n_0 such that:

$$g(n) \leq c.\, h(n)$$

for all $n \geq n_0$

<u>Example</u>:

There is a family $C_1, C_2, \ldots$ of Boolean circuits, where C_n adds two non-negative n-bits together such that,

$$\text{size}(C_n) = O(n)$$

This is equivalent to saying that the circuit has linear complexity relative to the size of the number of bits.

Multiplication: The Schonhage-Strassen algorithm reduces the computational complexity from $O(nm)$ to less.

Cost example:

- Addition of n bits: $O(n)$
- Multiplication of n bits: $O(n^2)$
- Division of n bits: $O(n^2)$
- GCD of n bits: $O(n^2)$
- Modular exponentiation for n bits: $O(n^3)$
- Integer factorization for n-bit numbers: $O(n^2 2^{n/2})$ (trial division)
- Integer factorization for n-bit numbers: $2^{O(n^{1/3} lg^{2/3}(n))}$ (number fields sieve)

The former have polynomial complexity, while trivial factorization for an integer has exponential complexity.

Polynomial vs. exponential cost:

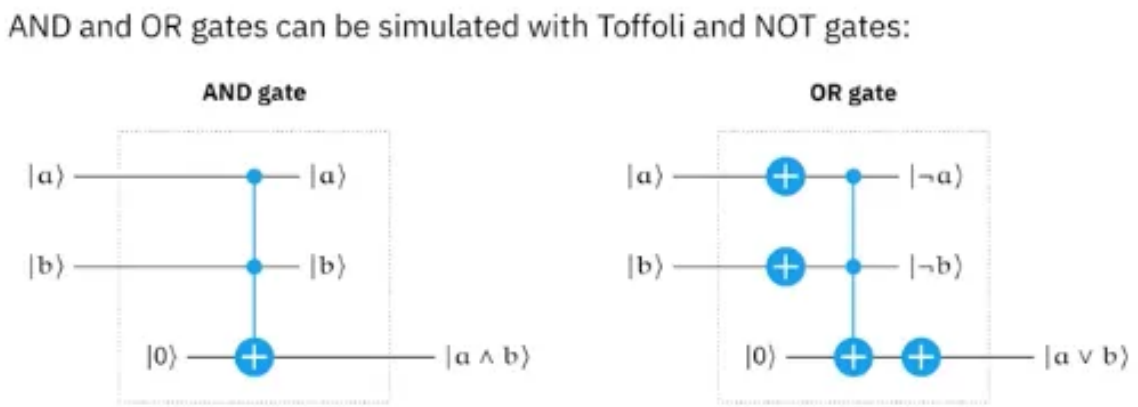
An algorithm has a polynomial cost if it has a complexity of $O(n^b)$ with $b > 0$

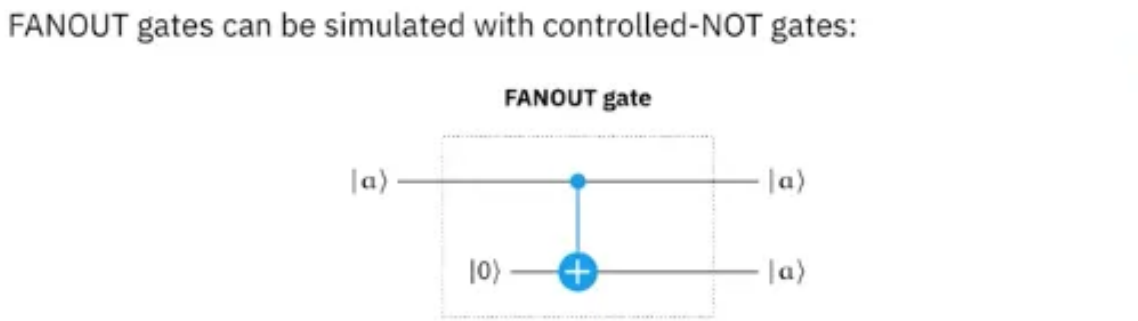
Examples: addition, multiplication, division, GCD, modular exponentiation

These are considered efficient algorithms. (Of course, a complexity such as $n^{1000000000}$ is not considered efficient).An algorithm has a sub-exponential cost if it has a complexity of $O(2^{n^c})$ for all $c > 0$. Otherwise, it is exponential (or super-exponential). There is no sub-exponential cost for classical integer factorization algorithms.Shor's algorithm is a quantum algorithm with a polynomial cost for integer factorization. NP-complete problems do not have sub-exponential complexity; the circuit is based on an exponential time formulation.

3. Classical Computations on quantum computers

AND and OR gates can be simulated using $TOFFOLI$ and NOT gates.

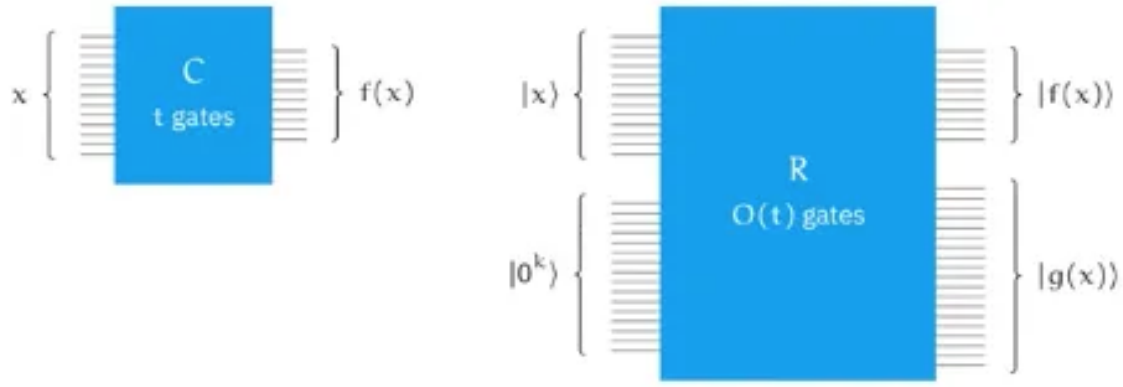




Suppose we have a Boolean circuit C of size t that performs a function:

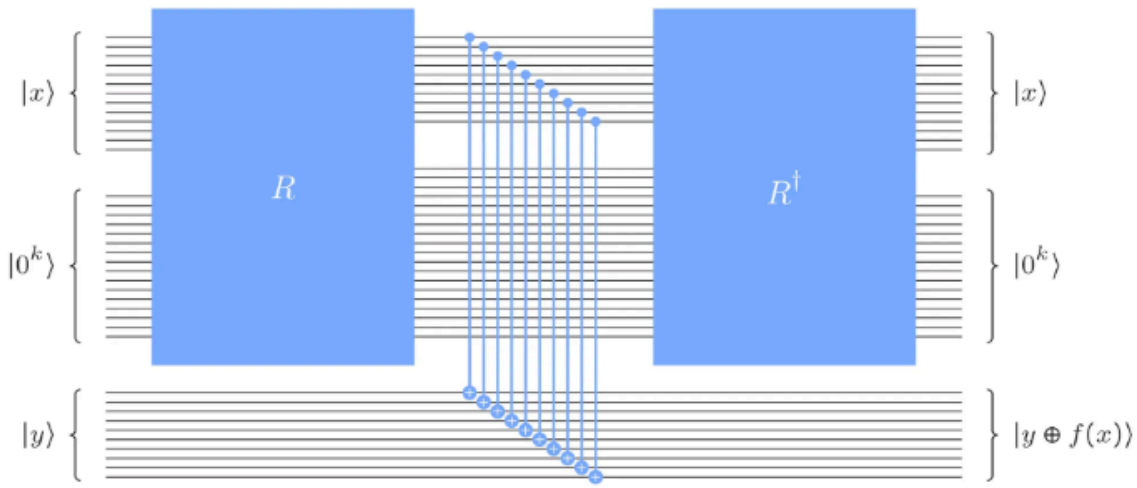
$$f : \Sigma^n \to \Sigma^m$$

By replacing each AND, OR, and $FANOUT$ gate in C with a quantum simulation, we obtain:

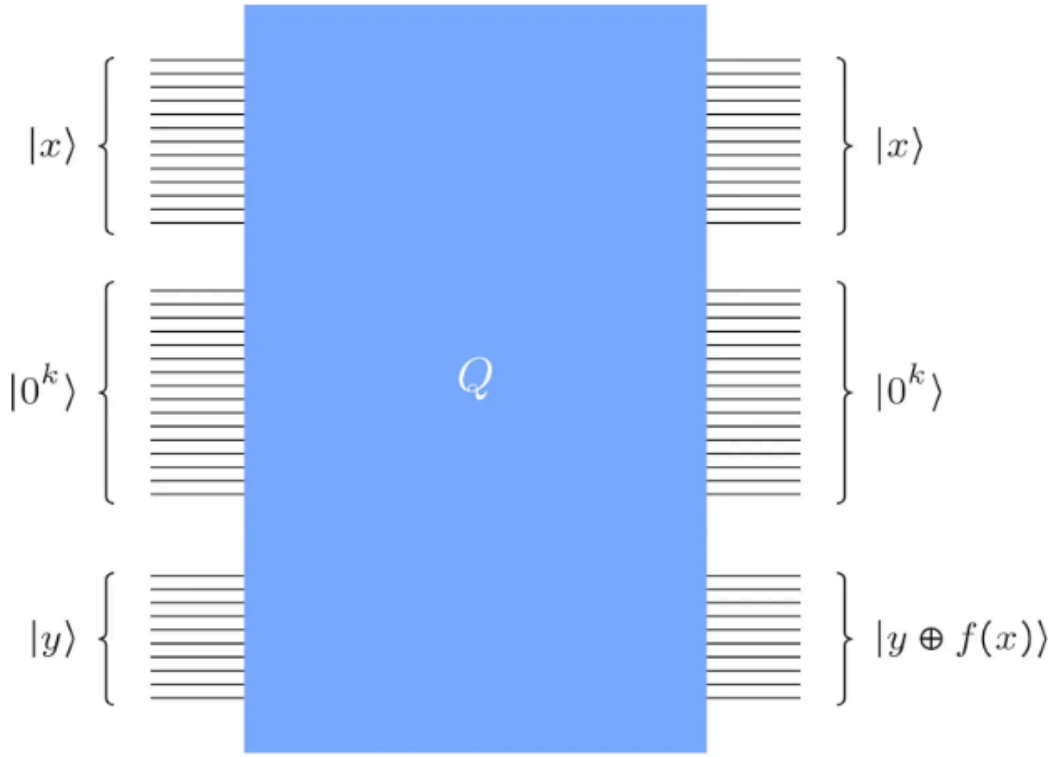


The input length of the quantum circuit is k qubits, so that the quantum gates can operate in C. The output consists of two functions, f and g. The function f corresponds to the output of the algorithm, and the function g (g as in garbage) corresponds to the useless output qubits following the calculation. The total number of gates for the quantum circuit is $O(t)$ because there is proportionality between the number of gates in the classical circuit and the quantum circuit.

However, it should be noted that these garbage qubits have an influence on the circuit, as interference is critical for quantum circuits, and garbage qubits can interfere with the result. In this case, to clean up the circuit, it must be inverted and applied after the quantum circuit. The result of f is recovered using a $CNOT$ gate on a chain of qubits outside the circuit.



We can note the final circuit Q, and we obtain the following diagram:


$|x\rangle$
$|0^k\rangle$
$|y\rangle$
Q
$|x\rangle$
$|0^k\rangle$
$|y \oplus f(x)\rangle$